

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software [1INF54-0982]

Estándar de Documentación de Código

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión del estándar de contenerización.	Equipo 6F

Contenido

1. Introducción	4
2. Uso obligatorio de JavaDoc	4
3. Comentarios de bloque para lógica compleja	5
4. Estructura requerida del README.md del repositorio principal	5
5. Idioma de los comentarios y la documentación	7
6. Documentación Viva de la API (OpenAPI/Swagger)	7
7. Registro de Decisiones Arquitectónicas (ADR)	7
8. Validación Automática en el Pipeline (CI)	7
9. Referencias y recursos	7

1. Introducción

Este documento define las reglas de comentarios internos y documentación del código fuente del sistema SysMile. Dado que el proyecto incluye algoritmos metaheurísticos (ALNS, IPSO) cuyo funcionamiento matemático no siempre es evidente a partir del código, la documentación adecuada de estos módulos es tan importante como la del resto del sistema.

2. Uso obligatorio de JavaDoc

Todo método público, y todo método privado que implemente un paso relevante de un algoritmo matemático, debe contar con un bloque JavaDoc que describa: propósito del método, parámetros (@param), valor de retorno (@return) y excepciones que puede lanzar (@throws).

Para los métodos que implementan pasos de ALNS o IPSO, el JavaDoc debe explicar además el fundamento matemático o heurístico del método (qué representa la operación en términos del algoritmo), no solo la firma técnica.

Java

```
/**
 * Aplica el operador de destrucción por remoción aleatoria sobre
 la solución actual,
 * eliminando un porcentaje configurable de paradas de la ruta
 para permitir su
 * posterior reinserción por el operador de reparación.
 *
 * Fundamento: introduce diversidad en el vecindario explorado
 por ALNS, evitando
 * que el algoritmo quede atrapado en óptimos locales.
 *
 * @param solution solución actual sobre la que se aplica la
 destrucción
 * @param destroyRate porcentaje de paradas a remover, en el
 rango (0, 1]
 * @return una nueva solución parcial con las paradas removidas
 * @throws IllegalArgumentException si destroyRate está fuera del
 rango permitido
 */
```

```
public PartialSolution destroy(Solution solution, double
destroyRate) { ... }
```

Las clases que representan un algoritmo o componente matemático (AlnsSolver, IpsoSwarm) deben incluir un JavaDoc de clase que cite la referencia bibliográfica o el paper en el que se basa la implementación, cuando aplique.

Generar la documentación JavaDoc como parte del pipeline de CI (mvn javadoc:javadoc) y publicarla como artefacto de referencia para el equipo, evitando que quede desactualizada respecto al código.

3. Comentarios de bloque para lógica compleja

- Usar comentarios de bloque (`/* ... */`) inmediatamente antes de cualquier sección de código cuya lógica no sea evidente por sí sola (ej. el cálculo de la función de fitness combinando distancia, tiempo y penalizaciones por ventana horaria).
- El comentario debe explicar el porqué de la decisión de diseño o la fórmula empleada, no una traducción literal de las líneas de código.

Java

```
/*
 * El costo total combina la distancia recorrida con una
 penalización cuadrática
 * por cada minuto fuera de la ventana horaria del cliente. Se
 eleva al cuadrado
 * para que retrasos grandes sean desproporcionadamente más
 costosos que varios
 * retrasos pequeños, favoreciendo soluciones más equilibradas.
 */
double cost = distance + Math.pow(lateMinutes, 2) *
PENALTY_FACTOR;
```

- Usar comentarios de línea (`//`) únicamente para aclaraciones breves y puntuales; si la explicación requiere más de dos líneas, debe convertirse en un comentario de bloque.
- En el frontend (TypeScript), aplicar el mismo criterio usando comentarios `/** JSDoc */` para funciones exportadas de hooks y utilidades, especialmente en el cálculo de métricas mostradas en el visualizador (ETA estimado, desviación de ruta).
- No dejar comentarios de código muerto (código comentado en lugar de eliminado); el historial de Git es la fuente de verdad para versiones anteriores del código.

4. Estructura requerida del README.md del repositorio principal

El archivo README.md ubicado en la raíz del repositorio debe mantener la siguiente estructura mínima, en este orden:

Sección	Contenido esperado
Título y descripción	Nombre del proyecto (SysMile), una o dos frases describiendo su propósito
Arquitectura	Diagrama o descripción breve de los componentes (backend, frontend, base de datos) y cómo se comunican
Requisitos previos	Versiones de Java, Node, PostgreSQL y Docker necesarias para ejecutar el proyecto
Instalación y ejecución local	Pasos para clonar, configurar variables de entorno (.env) y levantar el sistema con docker-compose
Estructura de carpetas	Referencia breve a la organización de backend/ y frontend/, enlazando a este conjunto de estándares
Ejecución de pruebas	Comandos para correr pruebas unitarias, de integración y end-to-end en cada módulo
Estándares del proyecto	Enlace a los documentos de estándares (61 a 69) para que cualquier nuevo integrante los ubique fácilmente
Convenciones de contribución	Referencia al estándar de Git (nombre de ramas, commits, pull requests)
Licencia y autores	Información de licencia del proyecto y créditos del equipo 6F

- El README.md debe mantenerse actualizado como parte de la definición de terminado (Definition of Done) de cualquier cambio que afecte la instalación, configuración o arquitectura del sistema.
- Los README.md específicos de cada módulo (backend/README.md, frontend/README.md) pueden profundizar en detalles propios de esa capa, pero deben enlazar de vuelta al README.md principal para evitar duplicación de información general.

5. Idioma de los comentarios y la documentación

- Los comentarios de código (JavaDoc, JSDoc, comentarios de bloque) se redactan en español, salvo los nombres de identificadores del código (clases, métodos, variables), que permanecen en inglés según las convenciones de nomenclatura definidas en los estándares de Java y Web.
- El README.md principal se redacta en español, siguiendo el idioma de trabajo del equipo.

6. Documentación Viva de la API (OpenAPI/Swagger)

- Todos los endpoints desarrollados en el backend deben estar anotados adecuadamente utilizando las anotaciones de OpenAPI (Swagger) para garantizar una interfaz interactiva y actualizada.
- La especificación OpenAPI generada debe incluir descripciones detalladas de los parámetros de solicitud, modelos de respuesta y códigos de estado HTTP esperados.

7. Registro de Decisiones Arquitectónicas (ADR)

- Cualquier cambio o decisión técnica relevante en la arquitectura del sistema debe documentarse formalmente mediante un Architectural Decision Record (ADR).
- Los archivos ADR se almacenarán en la carpeta /docs/adr del repositorio principal, manteniendo un formato estandarizado que contenga: título, contexto, decisión tomada y consecuencias.

8. Validación Automática en el Pipeline (CI)

- El pipeline de Integración Continua (CI) incluirá tareas automatizadas para verificar la existencia de JavaDoc/JSDoc en los métodos obligatorios antes de permitir la integración de cambios.

Los pull requests que no cumplan con la cobertura de documentación requerida o presenten errores en la generación de JavaDoc serán rechazados automáticamente.

9. Referencias y recursos

Guía oficial de JavaDoc: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>

Guía de JSDoc: <https://jsdoc.app/>

Make a README: <https://www.makeareadme.com/>